COMPUTER PROGRAMMING LAB

(23A05101P)

C Programs with Source Code and Output

B.Tech - Artificial Intelligence & Machine Learning
I Year - I Semester | JNTUA R23 Regulations

L-T-P-C : 0-0-3-1.5

Week	Topic	Programs
1	Programming Environment and First Program	2
2	Algorithms and Flowcharts	3
3	Variables, Data Types and Arithmetic Expressions	4
4	Operator Precedence and Associativity	6
5	Branching and Logical Expressions	5
6	Iterative Constructs	5
7	One-Dimensional Arrays and Searching	5
8	2-D Arrays, Strings and Sorting	6
9	Pointers, Structures and Dynamic Memory Allocation	5
10	Structures, Unions, Bit Fields and Linked Lists	5
11	Functions, Call by Value and Numerical Integration	4
12	Recursion	5
13	Pointers: Call by Reference and Dangling Pointers	4
14	File Handling	6
	Total programs	**65**

WEEK 1 : Programming Environment and First Program

Objective: Getting familiar with the programming environment on the computer and writing the first program.

Compile & run (Linux/gcc): gcc hello.c -o hello then ./hello (Turbo C: Alt+F9 to compile, Ctrl+F9 to run)

Program 1.1 : Simple program using printf()

Program:

```
#include <stdio.h>

int main()
{
    printf("Hello, World!\n");
    printf("Welcome to the C Programming Lab\n");
    return 0;
}
```

Output:

```
Hello, World!
Welcome to the C Programming Lab
```

Program 1.2 : Simple program using printf() and scanf()

Program:

```
#include <stdio.h>

int main()
{
    char name[30];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Enter your height (in feet): ");
    scanf("%f", &height);

    printf("\nName   : %s\n", name);
    printf("Age    : %d years\n", age);
    printf("Height : %.1f ft\n", height);
    return 0;
}
```

Output:

```
Enter your name: Ravi
Enter your age: 19
Enter your height (in feet): 5.8

Name   : Ravi
Age    : 19 years
Height : 5.8 ft
```

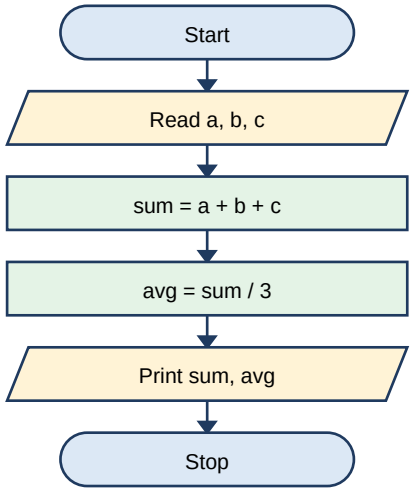
WEEK 2 : Algorithms and Flowcharts

Objective: Describe a solution to a problem in a series of finite steps using textual notation (algorithm) and graphic notation (flowchart), then convert it to C source code.

Program 2.1 : Sum and average of 3 numbers

Algorithm: 1. Start 2. Read a, b, c 3. sum = a + b + c 4. avg = sum / 3 5. Print sum, avg 6. Stop

Flowchart:



Program:

```
#include <stdio.h>

int main()
{
    float a, b, c, sum, avg;

    printf("Enter three numbers: ");
    scanf("%f %f %f", &a, &b, &c);

    sum = a + b + c;
    avg = sum / 3;

    printf("Sum     = %.2f\n", sum);
    printf("Average = %.2f\n", avg);
    return 0;
}
```

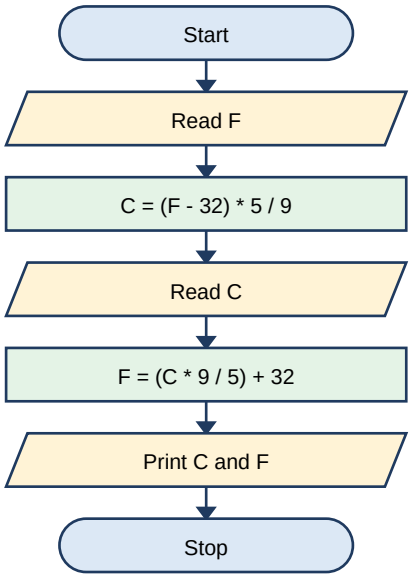
Output:

```
Enter three numbers: 10 20 35
Sum     = 65.00
Average = 21.67
```

Program 2.2 : Conversion of Fahrenheit to Celsius and vice versa

Algorithm: 1. Start 2. Read F 3. C = (F - 32) * 5 / 9 4. Read C 5. F = C * 9 / 5 + 32 6. Print results 7. Stop

Flowchart:



Program:

```
#include <stdio.h>

int main()
{
    float f, c;

    printf("Enter temperature in Fahrenheit: ");
    scanf("%f", &f);
    c = (f - 32) * 5 / 9;
    printf("%.2f F = %.2f C\n", f, c);

    printf("Enter temperature in Celsius: ");
    scanf("%f", &c);
    f = (c * 9 / 5) + 32;
    printf("%.2f C = %.2f F\n", c, f);
    return 0;
}
```

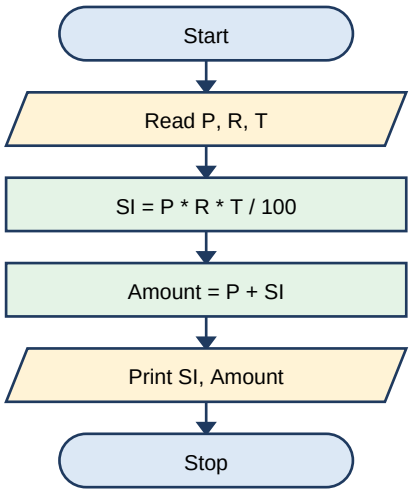
Output:

```
Enter temperature in Fahrenheit: 98.6
98.60 F = 37.00 C
Enter temperature in Celsius: 37
37.00 C = 98.60 F
```

Program 2.3 : Simple interest calculation

Algorithm: 1. Start 2. Read P, R, T 3. SI = P * R * T / 100 4. Amount = P + SI 5. Print SI, Amount 6. Stop

Flowchart:



Program:

```c
#include <stdio.h>

int main()
{
    float p, r, t, si, amount;

    printf("Enter principal, rate (%%) and time (years): ");
    scanf("%f %f %f", &p, &r, &t);

    si = (p * r * t) / 100;
    amount = p + si;

    printf("Simple Interest = %.2f\n", si);
    printf("Total Amount    = %.2f\n", amount);
    return 0;
}
```

Output:

```
Enter principal, rate (%) and time (years): 5000 8 3
Simple Interest = 1200.00
Total Amount    = 6200.00
```

WEEK 3 : Variables, Data Types and Arithmetic Expressions

Objective: Define variables with the desired data type, initialize them with appropriate values, and use arithmetic operators with variables and constants.

Program 3.1 : Finding the square root of a given number

Compile with math library: gcc sqrt.c -o sqrt -lm

Program:

```c
#include <stdio.h>
#include <math.h>

int main()
{
    double n;

    printf("Enter a number: ");
    scanf("%lf", &n);

    if (n < 0)
        printf("Square root of a negative number is not real\n");
    else
        printf("Square root of %.2f = %.4f\n", n, sqrt(n));
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter a number: 50
Square root of 50.00 = 7.0711

--- Test case 2 ---
Enter a number: -9
Square root of a negative number is not real
```

Program 3.2 : Finding compound interest

Program:

```
#include <stdio.h>
#include <math.h>

int main()
{
    float p, r, t, amount, ci;

    printf("Enter principal, rate (%%) and time (years): ");
    scanf("%f %f %f", &p, &r, &t);

    amount = p * pow(1 + r / 100, t);   /* A = P(1 + R/100)^T */
    ci = amount - p;

    printf("Compound Interest = %.2f\n", ci);
    printf("Total Amount      = %.2f\n", amount);
    return 0;
}
```

Output:

```
Enter principal, rate (%) and time (years): 10000 5 3
Compound Interest = 1576.25
Total Amount      = 11576.25
```

Program 3.3 : Area of a triangle using Heron's formula

Program:

```
#include <stdio.h>
#include <math.h>

int main()
{
    float a, b, c, s, area;

    printf("Enter the three sides of the triangle: ");
    scanf("%f %f %f", &a, &b, &c);

    s = (a + b + c) / 2;                          /* semi-perimeter */
    area = sqrt(s * (s - a) * (s - b) * (s - c));

    printf("Semi-perimeter = %.2f\n", s);
    printf("Area           = %.2f\n", area);
    return 0;
}
```

Output:

```
Enter the three sides of the triangle: 3 4 5
Semi-perimeter = 6.00
Area           = 6.00
```

Program 3.4 : Distance travelled by an object

Program:

```
#include <stdio.h>

int main()
{
    float u, a, t, s;

    printf("Enter initial velocity (m/s), acceleration (m/s^2), time (s): ");
    scanf("%f %f %f", &u, &a, &t);

    s = u * t + 0.5 * a * t * t;    /* s = ut + (1/2)at^2 */

    printf("Distance travelled = %.2f metres\n", s);
    return 0;
}
```

Output:

```
Enter initial velocity (m/s), acceleration (m/s^2), time (s): 10 2 5
Distance travelled = 75.00 metres
```

WEEK 4 : Operator Precedence and Associativity

Objective: Explore the full scope of expressions, type-compatibility of variables and constants, operators used in expressions and how operator precedence works.

Program 4.1 : Evaluate expression (a): A + B*C + (D*E) + F*G

Program:

```
#include <stdio.h>

int main()
{
    int A = 2, B = 3, C = 4, D = 5, E = 6, F = 7, G = 8, result;

    result = A + B * C + (D * E) + F * G;
    /* = 2 + 12 + 30 + 56 */
    printf("A + B*C + (D*E) + F*G = %d\n", result);
    return 0;
}
```

Output:

```
A + B*C + (D*E) + F*G = 100
```

Program 4.2 : Evaluate expression (b): A/B*C - B + A*D/3

Program:

```
#include <stdio.h>

int main()
{
    int A = 10, B = 2, C = 3, D = 6, result;

    /* (A/B)*C - B + (A*D)/3  ->  15 - 2 + 20 */
    result = A / B * C - B + A * D / 3;
    printf("A/B*C - B + A*D/3 = %d\n", result);
    return 0;
}
```

Output:

```
A/B*C - B + A*D/3 = 33
```

Program 4.3 : Evaluate expression (c): A+++B---A

Note: A is modified and read in the same expression, so the value is compiler dependent (unspecified behaviour).

Program:

```
#include <stdio.h>

int main()
{
    int A = 5, B = 3, result;

    /* Tokenised as (A++) + (B--) - A */
    result = A+++B---A;
    printf("Result = %d, A = %d, B = %d\n", result, A, B);
    return 0;
}
```

Output:

```
Result = 2, A = 6, B = 2
```

Program 4.4 : Evaluate expression (d): J = (i++) + (++i)

Note: i is modified twice without a sequence point, so the result is compiler dependent (undefined behaviour).

Program:

```
#include <stdio.h>

int main()
{
    int i = 5, J;

    J = (i++) + (++i);
    printf("i = %d, J = %d\n", i, J);
    return 0;
}
```

Output:

```
i = 7, J = 12
```

Program 4.5 : Maximum of three numbers using conditional operator

Program:

```
#include <stdio.h>

int main()
{
    int a, b, c, max;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);

    printf("Maximum = %d\n", max);
    return 0;
}
```

Output:

```
Enter three numbers: 12 45 23
Maximum = 45
```

Program 4.6 : Marks of 5 subjects: total and average (float)

Program:

```
#include <stdio.h>

int main()
{
    int m1, m2, m3, m4, m5, total;
    float average;

    printf("Enter marks of 5 subjects: ");
    scanf("%d %d %d %d %d", &m1, &m2, &m3, &m4, &m5);

    total = m1 + m2 + m3 + m4 + m5;
    average = total / 5.0;

    printf("Total   = %d\n", total);
    printf("Average = %.2f\n", average);
    return 0;
}
```

Output:

```
Enter marks of 5 subjects: 78 85 69 92 74
Total   = 398
Average = 79.60
```

WEEK 5 : Branching and Logical Expressions

Objective: Explore if-else, if-else-if ladder, switch and nested-if, along with relational and logical operators.

Program 5.1 : Maximum and minimum of four numbers using if-else

Program:

```
#include <stdio.h>

int main()
{
    int a, b, c, d, max, min;

    printf("Enter four numbers: ");
    scanf("%d %d %d %d", &a, &b, &c, &d);

    max = a;  min = a;
    if (b > max) max = b; else if (b < min) min = b;
    if (c > max) max = c; else if (c < min) min = c;
    if (d > max) max = d; else if (d < min) min = d;

    printf("Maximum = %d\n", max);
    printf("Minimum = %d\n", min);
    return 0;
}
```

Output:

```
Enter four numbers: 23 8 91 45
Maximum = 91
Minimum = 8
```

Program 5.2 : Electricity bill generation

Slabs: first 100 units Rs.1.5, next 100 Rs.2.5, next 100 Rs.4, above 300 Rs.5 per unit, plus Rs.50 fixed charge.

Program:

```
#include <stdio.h>

int main()
{
    int units;
    float bill, fixed = 50;

    printf("Enter units consumed: ");
    scanf("%d", &units);

    if (units <= 100)
        bill = units * 1.5;
    else if (units <= 200)
        bill = 100 * 1.5 + (units - 100) * 2.5;
    else if (units <= 300)
        bill = 100 * 1.5 + 100 * 2.5 + (units - 200) * 4;
    else
        bill = 100 * 1.5 + 100 * 2.5 + 100 * 4 + (units - 300) * 5;

    bill = bill + fixed;
    printf("Electricity bill = Rs. %.2f (including fixed charge Rs. %.0f)\n", bill, fixed);
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter units consumed: 80
Electricity bill = Rs. 170.00 (including fixed charge Rs. 50)

--- Test case 2 ---
Enter units consumed: 250
Electricity bill = Rs. 650.00 (including fixed charge Rs. 50)

--- Test case 3 ---
Enter units consumed: 350
Electricity bill = Rs. 1100.00 (including fixed charge Rs. 50)
```

Program 5.3 : Roots of a quadratic equation

Program:

```
#include <stdio.h>
#include <math.h>

int main()
{
    float a, b, c, d, r1, r2;

    printf("Enter coefficients a, b, c: ");
    scanf("%f %f %f", &a, &b, &c);

    d = b * b - 4 * a * c;                /* discriminant */

    if (d > 0) {
        r1 = (-b + sqrt(d)) / (2 * a);
        r2 = (-b - sqrt(d)) / (2 * a);
        printf("Roots are real and different: %.2f and %.2f\n", r1, r2);
    } else if (d == 0) {
        r1 = -b / (2 * a);
        printf("Roots are real and equal: %.2f and %.2f\n", r1, r1);
    } else {
        printf("Roots are complex: %.2f + %.2fi and %.2f - %.2fi\n",
               -b / (2 * a), sqrt(-d) / (2 * a), -b / (2 * a), sqrt(-d) / (2 * a));
    }
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter coefficients a, b, c: 1 -5 6
Roots are real and different: 3.00 and 2.00

--- Test case 2 ---
Enter coefficients a, b, c: 1 2 1
Roots are real and equal: -1.00 and -1.00

--- Test case 3 ---
Enter coefficients a, b, c: 1 2 5
Roots are complex: -1.00 + 2.00i and -1.00 - 2.00i
```

Program 5.4 : Calculator using switch case

Program:

```
#include <stdio.h>

int main()
{
    double a, b;
    char op;

    printf("Enter expression (e.g. 12 + 5): ");
    scanf("%lf %c %lf", &a, &op, &b);

    switch (op) {
        case '+': printf("%.2f + %.2f = %.2f\n", a, b, a + b); break;
        case '-': printf("%.2f - %.2f = %.2f\n", a, b, a - b); break;
        case '*': printf("%.2f * %.2f = %.2f\n", a, b, a * b); break;
        case '/':
            if (b == 0)
                printf("Error: division by zero\n");
            else
                printf("%.2f / %.2f = %.2f\n", a, b, a / b);
            break;
        default:  printf("Invalid operator\n");
    }
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter expression (e.g. 12 + 5): 12 + 5
12.00 + 5.00 = 17.00

--- Test case 2 ---
Enter expression (e.g. 12 + 5): 6 * 7
6.00 * 7.00 = 42.00

--- Test case 3 ---
Enter expression (e.g. 12 + 5): 9 / 0
Error: division by zero
```

Program 5.5 : Leap year or not

Program:

```
#include <stdio.h>

int main()
{
    int year;

    printf("Enter a year: ");
    scanf("%d", &year);

    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0))
        printf("%d is a leap year\n", year);
    else
        printf("%d is not a leap year\n", year);
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter a year: 2024
2024 is a leap year

--- Test case 2 ---
Enter a year: 1900
1900 is not a leap year

--- Test case 3 ---
Enter a year: 2000
2000 is a leap year
```

WEEK 6 : Iterative Constructs

Objective: Explore while, do-while and for loops, along with break and continue, and when each is most appropriate.

Program 6.1 : Factorial of a given number using a loop

Program:

```
#include <stdio.h>

int main()
{
    int n, i;
    unsigned long long fact = 1;

    printf("Enter a number: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
        fact = fact * i;

    printf("Factorial of %d = %llu\n", n, fact);
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter a number: 5
Factorial of 5 = 120

--- Test case 2 ---
Enter a number: 10
Factorial of 10 = 3628800
```

Program 6.2 : Check whether the given number is prime

Program:

```
#include <stdio.h>

int main()
{
    int n, i, flag = 0;

    printf("Enter a number: ");
    scanf("%d", &n);

    if (n < 2)
        flag = 1;
    for (i = 2; i <= n / 2; i++) {
        if (n % i == 0) {
            flag = 1;
            break;
        }
    }

    if (flag == 0)
        printf("%d is a prime number\n", n);
    else
        printf("%d is not a prime number\n", n);
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter a number: 29
29 is a prime number

--- Test case 2 ---
Enter a number: 15
15 is not a prime number
```

Program 6.3 : Compute sine and cosine series

Program:

```
#include <stdio.h>
#include <math.h>

int main()
{
    double x, rad, term, sinx, cosx;
    int i, n;

    printf("Enter angle in degrees and number of terms: ");
    scanf("%lf %d", &x, &n);
    rad = x * M_PI / 180;

    /* sin x = x - x^3/3! + x^5/5! - ... */
    term = rad;  sinx = rad;
    for (i = 1; i < n; i++) {
        term = -term * rad * rad / ((2 * i) * (2 * i + 1));
        sinx += term;
    }

    /* cos x = 1 - x^2/2! + x^4/4! - ... */
    term = 1;  cosx = 1;
    for (i = 1; i < n; i++) {
        term = -term * rad * rad / ((2 * i - 1) * (2 * i));
        cosx += term;
    }
    printf("sin(%.0f) by series = %.6f   (library: %.6f)\n", x, sinx, sin(rad));
```

```
    printf("cos(%.0f) by series = %.6f   (library: %.6f)\n", x, cosx, cos(rad));
    return 0;
}
```

Output:

```
Enter angle in degrees and number of terms: 30 10
sin(30) by series = 0.500000   (library: 0.500000)
cos(30) by series = 0.866025   (library: 0.866025)
```

Program 6.4 : Checking whether a number is a palindrome

Program:

```
#include <stdio.h>

int main()
{
    int n, temp, rev = 0, digit;

    printf("Enter a number: ");
    scanf("%d", &n);

    temp = n;
    while (temp > 0) {
        digit = temp % 10;
        rev = rev * 10 + digit;
        temp = temp / 10;
    }

    if (rev == n)
        printf("%d is a palindrome\n", n);
    else
        printf("%d is not a palindrome (reverse = %d)\n", n, rev);
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter a number: 12321
12321 is a palindrome

--- Test case 2 ---
Enter a number: 1234
1234 is not a palindrome (reverse = 4321)
```

Program 6.5 : Construct a pyramid of numbers

Program:

```
#include <stdio.h>

int main()
{
    int n, i, j, space;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (space = 1; space <= n - i; space++)
            printf(" ");
        for (j = 1; j <= i; j++)
            printf("%d ", j);
        printf("\n");
    }
    return 0;
}
```

Output:

```
Enter number of rows: 5
    1
   1 2
  1 2 3
 1 2 3 4
1 2 3 4 5
```

WEEK 7 : One-Dimensional Arrays and Searching

Objective: Define and initialize 1-D and 2-D arrays, reference individual elements, and use integer 1-D arrays to explore linear search.

Program 7.1 : Minimum and maximum of a 1-D integer array

Program:

```
#include <stdio.h>

int main()
{
    int a[50], n, i, min, max;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    min = max = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] > max) max = a[i];
        if (a[i] < min) min = a[i];
    }

    printf("Maximum = %d\nMinimum = %d\n", max, min);
    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter 6 elements: 34 7 89 12 56 3
Maximum = 89
Minimum = 3
```

Program 7.2 : Linear search on a 1-D array

Program:

```
#include <stdio.h>

int main()
{
    int a[50], n, i, key, found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter the element to search: ");
    scanf("%d", &key);

    for (i = 0; i < n; i++) {
        if (a[i] == key) {
            printf("%d found at position %d (index %d)\n", key, i + 1, i);
            found = 1;
            break;
        }
    }
    if (!found)
        printf("%d not found in the array\n", key);
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter number of elements: 6
Enter 6 elements: 34 7 89 12 56 3
Enter the element to search: 56
56 found at position 5 (index 4)

--- Test case 2 ---
Enter number of elements: 6
Enter 6 elements: 34 7 89 12 56 3
Enter the element to search: 100
100 not found in the array
```

Program 7.3 : Reverse of a 1-D integer array

Program:

```
#include <stdio.h>

int main()
{
    int a[50], n, i, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n / 2; i++) {          /* swap first and last, and so on */
        temp = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = temp;
    }

    printf("Reversed array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50
Reversed array: 50 40 30 20 10
```

Program 7.4 : 2's complement of a given binary number

Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char bin[33], ones[33], twos[33];
    int i, n, carry = 1;

    printf("Enter a binary number: ");
    scanf("%s", bin);
    n = strlen(bin);

    for (i = 0; i < n; i++)                  /* 1's complement */
        ones[i] = (bin[i] == '0') ? '1' : '0';
    ones[n] = '\0';

    strcpy(twos, ones);                      /* add 1 to get 2's complement */
    for (i = n - 1; i >= 0; i--) {
        if (twos[i] == '1' && carry)
            twos[i] = '0';
        else if (twos[i] == '0' && carry) {
            twos[i] = '1';
            carry = 0;
        }
    }
    printf("1's complement = %s\n", ones);
```

```
    printf("2's complement = %s\n", twos);
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter a binary number: 1011
1's complement = 0100
2's complement = 0101

--- Test case 2 ---
Enter a binary number: 01100100
1's complement = 10011011
2's complement = 10011100
```

Program 7.5 : Eliminate duplicate elements in an array

Program:

```
#include <stdio.h>

int main()
{
    int a[50], n, i, j, k;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; ) {
            if (a[i] == a[j]) {              /* shift left to remove duplicate */
                for (k = j; k < n - 1; k++)
                    a[k] = a[k + 1];
                n--;
            } else
                j++;
        }
    }

    printf("Array after removing duplicates: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Output:

```
Enter number of elements: 8
Enter 8 elements: 4 7 4 9 7 1 9 2
Array after removing duplicates: 4 7 9 1 2
```

WEEK 8 : 2-D Arrays, Strings and Sorting

Objective: Explore 2-D arrays, character arrays as strings (null terminated), string operations, and bubble sort on integer arrays.

Program 8.1 : Addition of two matrices

Program:

```
#include <stdio.h>

int main()
{
    int a[10][10], b[10][10], sum[10][10], r, c, i, j;

    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);

    printf("Enter elements of matrix A: ");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
    printf("Enter elements of matrix B: ");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &b[i][j]);

    printf("Sum of the matrices:\n");
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) {
            sum[i][j] = a[i][j] + b[i][j];
            printf("%4d", sum[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
Enter rows and columns: 2 3
Enter elements of matrix A: 1 2 3
4 5 6
Enter elements of matrix B: 6 5 4
3 2 1
Sum of the matrices:
   7   7   7
   7   7   7
```

Program 8.2 : Multiplication of two matrices

Program:

```
#include <stdio.h>

int main()
{
    int a[10][10], b[10][10], c[10][10] = {0};
    int r1, c1, r2, c2, i, j, k;

    printf("Enter rows and columns of A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns of B: ");
    scanf("%d %d", &r2, &c2);

    if (c1 != r2) {
        printf("Multiplication not possible (columns of A != rows of B)\n");
        return 0;
    }

    printf("Enter elements of A: ");
    for (i = 0; i < r1; i++)
        for (j = 0; j < c1; j++)
            scanf("%d", &a[i][j]);
    printf("Enter elements of B: ");
    for (i = 0; i < r2; i++)
        for (j = 0; j < c2; j++)
            scanf("%d", &b[i][j]);

    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++)
            for (k = 0; k < c1; k++)
                c[i][j] += a[i][k] * b[k][j];
    printf("Product matrix:\n");
```

```
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c2; j++)
            printf("%5d", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

Output:

```
Enter rows and columns of A: 2 3
Enter rows and columns of B: 3 2
Enter elements of A: 1 2 3
4 5 6
Enter elements of B: 7 8
9 10
11 12
Product matrix:
   58   64
  139  154
```

Program 8.3 : Sort array elements using bubble sort

Program:

```
#include <stdio.h>

int main()
{
    int a[50], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {            /* swap adjacent elements */
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter 6 elements: 64 25 12 22 11 90
Sorted array: 11 12 22 25 64 90
```

Program 8.4 : Concatenate two strings without built-in functions

Program:

```
#include <stdio.h>

int main()
{
    char s1[100], s2[50];
    int i = 0, j = 0;

    printf("Enter first string: ");
    scanf("%s", s1);
    printf("Enter second string: ");
    scanf("%s", s2);

    while (s1[i] != '\0')          /* move to end of s1 */
        i++;
    while (s2[j] != '\0') {        /* append s2 */
        s1[i] = s2[j];
        i++;
        j++;
    }
    s1[i] = '\0';

    printf("Concatenated string: %s\n", s1);
    return 0;
}
```

Output:

```
Enter first string: Hello
Enter second string: World
Concatenated string: HelloWorld
```

Program 8.5 : Reverse a string using built-in string functions

Note: strrev() is available only in Turbo C; with gcc, reverse using strlen()/strcpy() as shown.

Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[100], rev[100];
    int i, len;

    printf("Enter a string: ");
    scanf("%s", str);

    len = strlen(str);                      /* built-in function */
    for (i = 0; i < len; i++)
        rev[i] = str[len - 1 - i];
    rev[len] = '\0';

    strcpy(str, rev);                       /* built-in function */
    printf("Reversed string: %s\n", str);
    return 0;
}
```

Output:

```
Enter a string: programming
Reversed string: gnimmargorp
```

Program 8.6 : Reverse a string without using built-in string functions

Program:

```
#include <stdio.h>

int main()
{
    char str[100], temp;
    int i, len = 0;

    printf("Enter a string: ");
    scanf("%s", str);

    while (str[len] != '\0')                /* find length manually */
        len++;

    for (i = 0; i < len / 2; i++) {
        temp = str[i];
        str[i] = str[len - 1 - i];
        str[len - 1 - i] = temp;
    }

    printf("Reversed string: %s\n", str);
    return 0;
}
```

Output:

```
Enter a string: programming
Reversed string: gnimmargorp
```

WEEK 9 : Pointers, Structures and Dynamic Memory Allocation

Objective: Use pointers to manage dynamic arrays: malloc(), calloc(), realloc() and free(); process command-line arguments.

Program 9.1 : Sum of a 1-D array using malloc()

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p, n, i, sum = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    p = (int *)malloc(n * sizeof(int));
    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", p + i);
        sum += *(p + i);
    }

    printf("Sum of the elements = %d\n", sum);
    free(p);
    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50
Sum of the elements = 150
```

Program 9.2 : Total and average of n students using structures

Program:

```
#include <stdio.h>

struct student {
    char name[20];
    int m1, m2, m3, total;
    float avg;
};

int main()
{
    struct student s[10];
    int n, i;

    printf("Enter number of students: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter name and 3 marks of student %d: ", i + 1);
        scanf("%s %d %d %d", s[i].name, &s[i].m1, &s[i].m2, &s[i].m3);
        s[i].total = s[i].m1 + s[i].m2 + s[i].m3;
        s[i].avg = s[i].total / 3.0;
    }

    printf("\n%-10s %5s %8s\n", "Name", "Total", "Average");
    for (i = 0; i < n; i++)
        printf("%-10s %5d %8.2f\n", s[i].name, s[i].total, s[i].avg);
    return 0;
}
```

Output:

```
Enter number of students: 3
Enter name and 3 marks of student 1: Anil 80 75 90
Enter name and 3 marks of student 2: Bhavya 65 70 72
Enter name and 3 marks of student 3: Chandu 90 88 95

Name       Total  Average
Anil         245    81.67
Bhavya       207    69.00
Chandu       273    91.00
```

Program 9.3 : Enter n students' data using calloc() and display failed students

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct student {
    int roll;
    char name[20];
    float marks;
};

int main()
{
    struct student *s;
    int n, i, count = 0;

    printf("Enter number of students: ");
    scanf("%d", &n);

    s = (struct student *)calloc(n, sizeof(struct student));
    if (s == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        printf("Enter roll no, name and marks of student %d: ", i + 1);
        scanf("%d %s %f", &s[i].roll, s[i].name, &s[i].marks);
    }

    printf("\nFailed students (marks < 40):\n");
    for (i = 0; i < n; i++) {
```

```
        if (s[i].marks < 40) {
            printf("%d  %-10s %.1f\n", s[i].roll, s[i].name, s[i].marks);
            count++;
        }
    }
    if (count == 0)
        printf("None. All students passed.\n");

    free(s);
    return 0;
}
```

Output:

```
Enter number of students: 4
Enter roll no, name and marks of student 1: 101 Ravi 78
Enter roll no, name and marks of student 2: 102 Sita 35
Enter roll no, name and marks of student 3: 103 Kiran 40
Enter roll no, name and marks of student 4: 104 Meena 28

Failed students (marks < 40):
102  Sita       35.0
104  Meena      28.0
```

Program 9.4 : Student name and marks from the command line, with total

Run as: ./student Ravi 78 85 90

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int i, total = 0;

    if (argc < 3) {
        printf("Usage: %s name mark1 mark2 ...\n", argv[0]);
        return 1;
    }

    printf("Student name : %s\n", argv[1]);
    for (i = 2; i < argc; i++) {
        printf("Subject %d    : %d\n", i - 1, atoi(argv[i]));
        total += atoi(argv[i]);
    }
    printf("Total marks  : %d\n", total);
    printf("Average      : %.2f\n", (float)total / (argc - 2));
    return 0;
}
```

Output:

```
$ ./a.out Ravi 78 85 90
Student name : Ravi
Subject 1    : 78
Subject 2    : 85
Subject 3    : 90
Total marks  : 253
Average      : 84.33
```

Program 9.5 : Implementing realloc()

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p, i;

    p = (int *)malloc(3 * sizeof(int));          /* initial size = 3 */
    for (i = 0; i < 3; i++)
        p[i] = (i + 1) * 10;

    printf("Before realloc: ");
    for (i = 0; i < 3; i++)
        printf("%d ", p[i]);

    p = (int *)realloc(p, 6 * sizeof(int));      /* resize to 6 */
    if (p == NULL) {
        printf("\nrealloc failed\n");
        return 1;
    }
    for (i = 3; i < 6; i++)
        p[i] = (i + 1) * 10;

    printf("\nAfter realloc (size 6): ");
    for (i = 0; i < 6; i++)
        printf("%d ", p[i]);

    p = (int *)realloc(p, 2 * sizeof(int));      /* shrink to 2 */
    printf("\nAfter shrinking (size 2): ");
    for (i = 0; i < 2; i++)
        printf("%d ", p[i]);
    printf("\n");

    free(p);
    return 0;
}
```

Output:

```
Before realloc: 10 20 30
After realloc (size 6): 10 20 30 40 50 60
After shrinking (size 2): 10 20
```

WEEK 10 : Structures, Unions, Bit Fields and Linked Lists

Objective: Experiment with C structures, unions, bit fields, self-referential structures (singly linked lists) and nested structures.

Program 10.1 : Read and print a date (dd/mm/yyyy) using bit-fields and without bit-fields

Program:

```
#include <stdio.h>

struct date_bf {                 /* with bit-fields */
    unsigned int day   : 5;      /* 1..31   */
    unsigned int month : 4;      /* 1..12   */
    unsigned int year  : 12;     /* 0..4095 */
};

struct date_nb {                 /* without bit-fields */
    int day, month, year;
};

int main()
{
    int d, m, y;
    struct date_bf a;
    struct date_nb b;

    printf("Enter date (dd/mm/yyyy): ");
    scanf("%d/%d/%d", &d, &m, &y);

    a.day = d;  a.month = m;  a.year = y;
```

```
    b.day = d;  b.month = m;  b.year = y;

    printf("Using bit-fields     : %02d/%02d/%04d   size = %lu bytes\n",
           a.day, a.month, a.year, sizeof(a));
    printf("Without bit-fields   : %02d/%02d/%04d   size = %lu bytes\n",
           b.day, b.month, b.year, sizeof(b));
    return 0;
}
```

Output:

```
Enter date (dd/mm/yyyy): 15/08/2025
Using bit-fields     : 15/08/2025   size = 4 bytes
Without bit-fields   : 15/08/2025   size = 12 bytes
```

Program 10.2 : Create and display a singly linked list (self-referential structure)

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;           /* self-referential pointer */
};

int main()
{
    struct node *head = NULL, *temp, *last = NULL;
    int n, i, val;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter data for node %d: ", i + 1);
        scanf("%d", &val);
        temp = (struct node *)malloc(sizeof(struct node));
        temp->data = val;
        temp->next = NULL;
        if (head == NULL)
            head = temp;
        else
            last->next = temp;
        last = temp;
    }

    printf("Linked list: ");
    for (temp = head; temp != NULL; temp = temp->next)
        printf("%d -> ", temp->data);
    printf("NULL\n");

    while (head != NULL) {       /* free all nodes */
        temp = head;
        head = head->next;
        free(temp);
    }
    return 0;
}
```

Output:

```
Enter number of nodes: 4
Enter data for node 1: 10
Enter data for node 2: 20
Enter data for node 3: 30
Enter data for node 4: 40
Linked list: 10 -> 20 -> 30 -> 40 -> NULL
```

Program 10.3 : Differences between structures and unions

Program:

```
#include <stdio.h>

struct s_data { int i; float f; char c; };
union  u_data { int i; float f; char c; };

int main()
{
    struct s_data s;
    union  u_data u;

    printf("Size of structure = %lu bytes\n", sizeof(s));
    printf("Size of union     = %lu bytes\n\n", sizeof(u));

    s.i = 65;  s.f = 3.14;  s.c = 'A';
    printf("Structure -> i = %d, f = %.2f, c = %c  (all members kept)\n", s.i, s.f, s.c);

    u.i = 65;
    printf("Union     -> after u.i = 65   : i = %d\n", u.i);
    u.f = 3.14;
    printf("Union     -> after u.f = 3.14 : f = %.2f, i = %d (overwritten)\n", u.f, u.i);
    u.c = 'A';
    printf("Union     -> after u.c = 'A'  : c = %c\n", u.c);
    return 0;
}
```

Output:

```
Size of structure = 12 bytes
Size of union     = 4 bytes

Structure -> i = 65, f = 3.14, c = A  (all members kept)
Union     -> after u.i = 65   : i = 65
Union     -> after u.f = 3.14 : f = 3.14, i = 1078523331 (overwritten)
Union     -> after u.c = 'A'  : c = A
```

Program 10.4 : Shift / rotate using bit-fields

Program:

```
#include <stdio.h>

struct reg { unsigned int bits : 8; };      /* 8-bit register */

void show(const char *msg, unsigned int v)
{
    int i;
    printf("%-18s: ", msg);
    for (i = 7; i >= 0; i--)
        printf("%d", (v >> i) & 1);
    printf("  (%u)\n", v);
}

int main()
{
    struct reg r, t;
    unsigned int val, n;

    printf("Enter an 8-bit value (0-255) and shift count: ");
    scanf("%u %u", &val, &n);

    r.bits = val;
    show("Original", r.bits);

    t.bits = r.bits << n;
    show("Shift left", t.bits);

    t.bits = r.bits >> n;
    show("Shift right", t.bits);

    t.bits = (r.bits << n) | (r.bits >> (8 - n));
    show("Rotate left", t.bits);

    t.bits = (r.bits >> n) | (r.bits << (8 - n));
    show("Rotate right", t.bits);
    return 0;
```

```
}
```

Output:

```
Enter an 8-bit value (0-255) and shift count: 181 2
Original          : 10110101  (181)
Shift left        : 11010100  (212)
Shift right       : 00101101  (45)
Rotate left       : 11010110  (214)
Rotate right      : 01101101  (109)
```

Program 10.5 : Copy one structure variable to another of the same type

Program:

```
#include <stdio.h>
#include <string.h>

struct student {
    int roll;
    char name[20];
    float marks;
};

int main()
{
    struct student s1 = {101, "Ravi", 86.5}, s2, s3;

    s2 = s1;                            /* direct structure assignment */
    memcpy(&s3, &s1, sizeof(s1));       /* using memcpy */

    strcpy(s2.name, "Sita");            /* modify copy: original unaffected */
    s2.roll = 102;

    printf("s1 : %d  %-6s %.1f\n", s1.roll, s1.name, s1.marks);
    printf("s2 : %d  %-6s %.1f\n", s2.roll, s2.name, s2.marks);
    printf("s3 : %d  %-6s %.1f\n", s3.roll, s3.name, s3.marks);
    return 0;
}
```

Output:

```
s1 : 101  Ravi   86.5
s2 : 102  Sita   86.5
s3 : 101  Ravi   86.5
```

WEEK 11 : Functions, Call by Value and Numerical Integration

Objective: Explore functions, sub-routines, scope and extent of variables, parameter passing by value, and basic numerical integration.

Program 11.1 : Function to calculate nCr

Program:

```
#include <stdio.h>

long fact(int n)
{
    long f = 1;
    int i;
    for (i = 2; i <= n; i++)
        f *= i;
    return f;
}

long ncr(int n, int r)
{
    return fact(n) / (fact(r) * fact(n - r));
}

int main()
{
    int n, r;

    printf("Enter n and r: ");
    scanf("%d %d", &n, &r);
    if (r > n)
```

```
        printf("Invalid input: r must not exceed n\n");
    else
        printf("%dC%d = %ld\n", n, r, ncr(n, r));
    return 0;
}
```

Output:

```
Enter n and r: 10 3
10C3 = 120
```

Program 11.2 : Function to find the length of a string

Program:

```
#include <stdio.h>

int string_length(char s[])
{
    int i = 0;
    while (s[i] != '\0')
        i++;
    return i;
}

int main()
{
    char str[100];
    int i = 0;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    while (str[i] != '\n' && str[i] != '\0') i++;
    str[i] = '\0';                          /* remove trailing newline */

    printf("Length of \"%s\" = %d\n", str, string_length(str));
    return 0;
}
```

Output:

```
Enter a string: Computer Programming
Length of "Computer Programming" = 20
```

Program 11.3 : Function to transpose a matrix

Program:

```
#include <stdio.h>

void transpose(int a[10][10], int t[10][10], int r, int c)
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            t[j][i] = a[i][j];
}

int main()
{
    int a[10][10], t[10][10], r, c, i, j;

    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);
    printf("Enter matrix elements: ");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);

    transpose(a, t, r, c);

    printf("Transpose of the matrix:\n");
    for (i = 0; i < c; i++) {
        for (j = 0; j < r; j++)
            printf("%4d", t[i][j]);
        printf("\n");
    }
    return 0;
}
```

Output:

```
Enter rows and columns: 2 3
Enter matrix elements: 1 2 3
4 5 6
Transpose of the matrix:
   1   4
   2   5
   3   6
```

Program 11.4 : Numerical integration of a differential equation using Euler's method

Program:

```
#include <stdio.h>
#include <math.h>

/* dy/dx = x + y,  y(0) = 1   (exact solution: y = 2e^x - x - 1) */
double f(double x, double y)
{
    return x + y;
}

int main()
{
    double x0, y0, h, xn, x, y;
    int i, n;

    printf("Enter x0, y0, step size h and final x: ");
    scanf("%lf %lf %lf %lf", &x0, &y0, &h, &xn);

    n = (int)((xn - x0) / h + 0.5);
    x = x0;  y = y0;

    printf("\n%6s %10s %12s\n", "x", "y (Euler)", "y (exact)");
    printf("%6.2f %10.5f %12.5f\n", x, y, 2 * exp(x) - x - 1);
    for (i = 1; i <= n; i++) {
        y = y + h * f(x, y);              /* y(n+1) = y(n) + h f(xn, yn) */
        x = x0 + i * h;
        printf("%6.2f %10.5f %12.5f\n", x, y, 2 * exp(x) - x - 1);
    }
    return 0;
}
```

Output:

```
Enter x0, y0, step size h and final x: 0 1 0.1 0.5

     x  y (Euler)    y (exact)
  0.00    1.00000      1.00000
  0.10    1.10000      1.11034
  0.20    1.22000      1.24281
  0.30    1.36200      1.39972
  0.40    1.52820      1.58365
  0.50    1.72102      1.79744
```

WEEK 12 : Recursion

Objective: Write recursive functions invoked from main for problems that have naturally recursive solutions.

Program 12.1 : Recursive function to generate Fibonacci series

Program:

```
#include <stdio.h>

int fib(int n)
{
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}

int main()
{
    int n, i;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci series: ");
    for (i = 0; i < n; i++)
        printf("%d ", fib(i));
    printf("\n");
    return 0;
}
```

Output:

```
Enter number of terms: 10
Fibonacci series: 0 1 1 2 3 5 8 13 21 34
```

Program 12.2 : Recursive function to find the LCM of two numbers

Program:

```
#include <stdio.h>

/* tries successive multiples of the larger number */
int lcm(int a, int b, int multiple)
{
    int step = (a > b) ? a : b;

    if (multiple % a == 0 && multiple % b == 0)
        return multiple;
    return lcm(a, b, multiple + step);
}

int main()
{
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("LCM of %d and %d = %d\n", a, b, lcm(a, b, (a > b) ? a : b));
    return 0;
}
```

Output:

```
Enter two numbers: 12 18
LCM of 12 and 18 = 36
```

Program 12.3 : Recursive function to find the factorial of a number

Program:

```
#include <stdio.h>

long fact(int n)
{
    if (n == 0 || n == 1)
        return 1;
    return n * fact(n - 1);
}

int main()
{
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    printf("Factorial of %d = %ld\n", n, fact(n));
    return 0;
}
```

Output:

```
Enter a number: 6
Factorial of 6 = 720
```

Program 12.4 : Ackermann function using recursion

Program:

```
#include <stdio.h>

int ackermann(int m, int n)
{
    if (m == 0)
        return n + 1;
    if (n == 0)
        return ackermann(m - 1, 1);
    return ackermann(m - 1, ackermann(m, n - 1));
}

int main()
{
    int m, n;

    printf("Enter m and n: ");
    scanf("%d %d", &m, &n);

    printf("Ackermann(%d, %d) = %d\n", m, n, ackermann(m, n));
    return 0;
}
```

Output:

```
--- Test case 1 ---
Enter m and n: 2 3
Ackermann(2, 3) = 9

--- Test case 2 ---
Enter m and n: 3 2
Ackermann(3, 2) = 29
```

Program 12.5 : Recursive function to find the sum of a series 1 + 1/2 + 1/3 + ... + 1/n

Program:

```
#include <stdio.h>

double sum_series(int n)
{
    if (n == 1)
        return 1.0;
    return 1.0 / n + sum_series(n - 1);
}

int main()
{
    int n;

    printf("Enter n: ");
    scanf("%d", &n);

    printf("Sum of the series up to 1/%d = %.4f\n", n, sum_series(n));
    return 0;
}
```

Output:

```
Enter n: 5
Sum of the series up to 1/5 = 2.2833
```

WEEK 13 : Pointers: Call by Reference and Dangling Pointers

Objective: Basic difference between normal and pointer variables, arithmetic operations using pointers, and passing variables to functions using pointers.

Program 13.1 : Swap two numbers using call by reference

Program:

```
#include <stdio.h>

void swap(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}

int main()
{
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("Before swapping: a = %d, b = %d\n", a, b);
    swap(&a, &b);
    printf("After swapping : a = %d, b = %d\n", a, b);
    return 0;
}
```

Output:

```
Enter two numbers: 15 40
Before swapping: a = 15, b = 40
After swapping : a = 40, b = 15
```

Program 13.2 : Dangling pointer problem demonstration

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p = (int *)malloc(sizeof(int));
    int *q;

    *p = 25;
    printf("Value at p before free : %d\n", *p);

    free(p);                        /* memory released, but p still holds the old address */
    if (p != NULL)
        printf("After free(p): p is NOT NULL -> p is a dangling pointer\n");
    /* printf("%d", *p);  <-- undefined behaviour, never do this */

    p = NULL;                       /* fix: set pointer to NULL after free */
    printf("After p = NULL: p is %s\n", (p == NULL) ? "NULL (safe)" : "not NULL");

    {
        int y = 99;
        q = &y;
        printf("Inside block : *q = %d\n", *q);
    }                               /* y goes out of scope here: q is now dangling */
    printf("Outside block: q still points to a dead variable (do not use *q)\n");
    q = NULL;
    return 0;
}
```

Output:

```
Value at p before free : 25
After free(p): p is NOT NULL -> p is a dangling pointer
After p = NULL: p is NULL (safe)
Inside block : *q = 99
Outside block: q still points to a dead variable (do not use *q)
```

Program 13.3 : Copy one string into another using pointers

Program:

```
#include <stdio.h>

void string_copy(char *dest, char *src)
{
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0';
}

int main()
{
    char source[100], target[100];
    int i = 0;

    printf("Enter a string: ");
    fgets(source, sizeof(source), stdin);
    while (source[i] != '\n' && source[i] != '\0') i++;
    source[i] = '\0';

    string_copy(target, source);
    printf("Source string : %s\n", source);
    printf("Copied string : %s\n", target);
    return 0;
}
```

Output:

```
Enter a string: Hello Pointers
Source string : Hello Pointers
Copied string : Hello Pointers
```

Program 13.4 : Count lowercase, uppercase, digits and other characters using pointers

Program:

```
#include <stdio.h>

int main()
{
    char str[100], *p;
    int lower = 0, upper = 0, digits = 0, other = 0;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    for (p = str; *p != '\0' && *p != '\n'; p++) {
        if (*p >= 'a' && *p <= 'z')
            lower++;
        else if (*p >= 'A' && *p <= 'Z')
            upper++;
        else if (*p >= '0' && *p <= '9')
            digits++;
        else
            other++;
    }

    printf("Lowercase letters : %d\n", lower);
    printf("Uppercase letters : %d\n", upper);
    printf("Digits            : %d\n", digits);
    printf("Other characters  : %d\n", other);
    return 0;
}
```

Output:

```
Enter a string: Hello World 2026 @C!
Lowercase letters : 8
Uppercase letters : 3
Digits            : 4
Other characters  : 5
```

WEEK 14 : File Handling

Objective: Understand data files and file handling with various file I/O functions; explore the differences between text and binary files.

Program 14.1 : Write and read text in a file

Program:

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char text[100], ch;

    printf("Enter a line of text: ");
    fgets(text, sizeof(text), stdin);

    fp = fopen("sample.txt", "w");          /* write mode */
    if (fp == NULL) {
        printf("Cannot open file\n");
        return 1;
    }
    fputs(text, fp);
    fclose(fp);
    printf("Text written to sample.txt\n");

    fp = fopen("sample.txt", "r");          /* read mode */
    printf("File contents: ");
    while ((ch = fgetc(fp)) != EOF)
        putchar(ch);
    fclose(fp);
    return 0;
}
```

Output:

```
Enter a line of text: Learning file handling in C is fun
Text written to sample.txt
File contents: Learning file handling in C is fun
```

Program 14.2 : Write and read a binary file using fwrite() and fread()

Program:

```
#include <stdio.h>

struct student {
    int roll;
    char name[20];
    float marks;
};

int main()
{
    struct student w[3] = {{101, "Ravi", 78.5}, {102, "Sita", 91.0}, {103, "Kiran", 66.25}};
    struct student r;
    FILE *fp;
    int i;

    fp = fopen("students.dat", "wb");
    fwrite(w, sizeof(struct student), 3, fp);
    fclose(fp);
    printf("3 records written to students.dat\n");

    fp = fopen("students.dat", "rb");
    printf("\nRecords read from the binary file:\n");
    for (i = 0; i < 3; i++) {
        fread(&r, sizeof(struct student), 1, fp);
        printf("%d  %-8s %.2f\n", r.roll, r.name, r.marks);
    }
    fseek(fp, 0, SEEK_END);
    printf("\nFile size = %ld bytes\n", ftell(fp));
    fclose(fp);
    return 0;
}
```

Output:

```
3 records written to students.dat

Records read from the binary file:
101  Ravi     78.50
102  Sita     91.00
103  Kiran    66.25

File size = 84 bytes
```

Program 14.3 : Copy the contents of one file to another file

```
Input file a.txt:
Welcome to C programming lab.
File handling is fun.
Keep practising every day.
```

Program:

```
#include <stdio.h>

int main()
{
    FILE *src, *dest;
    char sname[30], dname[30], ch;

    printf("Enter source file name: ");
    scanf("%s", sname);
    printf("Enter destination file name: ");
    scanf("%s", dname);

    src = fopen(sname, "r");
    if (src == NULL) {
        printf("Source file not found\n");
        return 1;
    }
```

```
    dest = fopen(dname, "w");

    while ((ch = fgetc(src)) != EOF)
        fputc(ch, dest);
    fclose(src);
    fclose(dest);
    printf("File copied successfully\n\nContents of %s:\n", dname);

    dest = fopen(dname, "r");
    while ((ch = fgetc(dest)) != EOF)
        putchar(ch);
    fclose(dest);
    return 0;
}
```

Output:

```
Enter source file name: a.txt
Enter destination file name: copy.txt
File copied successfully

Contents of copy.txt:
Welcome to C programming lab.
File handling is fun.
Keep practising every day.
```

Program 14.4 : Merge two files into a third file using command-line arguments

```
Run as:  ./merge a.txt b.txt merged.txt

a.txt:
Welcome to C programming lab.
File handling is fun.
Keep practising every day.

b.txt:
This is the second file.
It will be merged with the first.
```

Program:

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    FILE *f1, *f2, *f3;
    char ch;

    if (argc != 4) {
        printf("Usage: %s file1 file2 outputfile\n", argv[0]);
        return 1;
    }

    f1 = fopen(argv[1], "r");
    f2 = fopen(argv[2], "r");
    if (f1 == NULL || f2 == NULL) {
        printf("Unable to open input files\n");
        return 1;
    }
    f3 = fopen(argv[3], "w");

    while ((ch = fgetc(f1)) != EOF)
        fputc(ch, f3);
    while ((ch = fgetc(f2)) != EOF)
        fputc(ch, f3);
    fclose(f1);  fclose(f2);  fclose(f3);
    printf("Files merged into %s\n\nContents of %s:\n", argv[3], argv[3]);

    f3 = fopen(argv[3], "r");
    while ((ch = fgetc(f3)) != EOF)
        putchar(ch);
    fclose(f3);
    return 0;
}
```

Output:

```
$ ./a.out a.txt b.txt merged.txt
Files merged into merged.txt

Contents of merged.txt:
Welcome to C programming lab.
File handling is fun.
Keep practising every day.
This is the second file.
It will be merged with the first.
```

Program 14.5 : Find number of lines, words and characters in a file

```
Input file a.txt:
Welcome to C programming lab.
File handling is fun.
Keep practising every day.
```

Program:

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char fname[30], ch;
    int lines = 0, words = 0, chars = 0, inword = 0;

    printf("Enter file name: ");
    scanf("%s", fname);

    fp = fopen(fname, "r");
    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF) {
        chars++;
        if (ch == '\n')
            lines++;
        if (ch == ' ' || ch == '\n' || ch == '\t')
            inword = 0;
        else if (inword == 0) {
            inword = 1;
            words++;
        }
    }
    fclose(fp);

    printf("Lines      = %d\n", lines);
    printf("Words      = %d\n", words);
    printf("Characters = %d\n", chars);
    return 0;
}
```

Output:

```
Enter file name: a.txt
Lines      = 3
Words      = 13
Characters = 79
```

Program 14.6 : Print the last n characters of a given file

```
Input file a.txt:
Welcome to C programming lab.
File handling is fun.
Keep practising every day.
```

Program:

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char fname[30], ch;
    int n;
    long size;

    printf("Enter file name and n: ");
    scanf("%s %d", fname, &n);

    fp = fopen(fname, "r");
    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }

    fseek(fp, 0, SEEK_END);
    size = ftell(fp);                       /* total characters in file */
    if (n > size)
        n = size;
    fseek(fp, -n, SEEK_END);                /* move n characters back from end */

    printf("Last %d characters:\n", n);
    while ((ch = fgetc(fp)) != EOF)
        putchar(ch);
    fclose(fp);
    return 0;
}
```

Output:

```
Enter file name and n: a.txt 27
Last 27 characters:
Keep practising every day.
```